

**Universidad de San Carlos de Guatemala**

Facultad de Ingeniería

Sistemas de Bases de Datos 2

# **Administración de un DBMS**

Tareas críticas, riesgos y resolución de problemas

## **Trabajo Práctico No. 01**

**Estudiante:** Santiago Julián Barrera Reyes

**Carné:** 201905884

**Curso:** Sistemas de Bases de Datos 2

**Catedrático:** Ing. Marlon Francisco Orellana López

Guatemala, 6 de junio de 2026

## Índice

|                                                                                                                  |    |
|------------------------------------------------------------------------------------------------------------------|----|
| Índice.....                                                                                                      | 2  |
| 1. INTRODUCCIÓN .....                                                                                            | 3  |
| 2. SECCIÓN I — TAREAS CRÍTICAS QUE UN DBA DEBE EJECUTAR REGULARMENTE.....                                        | 3  |
| 2.1. Tarea 1 — Respaldo y verificación de copias de seguridad (Backup & Recovery) .....                          | 3  |
| 2.2. Tarea 2 — Monitoreo de rendimiento y optimización de consultas .....                                        | 4  |
| 2.3. Tarea 3 — Mantenimiento de índices y estadísticas .....                                                     | 5  |
| 2.4. Tarea 4 — Gestión de seguridad, control de accesos y auditoría .....                                        | 5  |
| 2.5. Tarea 5 — Gestión de capacidad y crecimiento del almacenamiento.....                                        | 6  |
| 3. SECCIÓN II — RIESGOS SIGNIFICATIVOS EN LA ADMINISTRACIÓN DE UN DBMS ..                                        | 7  |
| 3.1. Riesgo 1 — Pérdida de datos por fallo o ausencia de respaldo válido .....                                   | 7  |
| 3.2. Riesgo 2 — Tiempo de inactividad (downtime) no planificado .....                                            | 7  |
| 3.3. Riesgo 3 — Brecha de seguridad y acceso no autorizado .....                                                 | 8  |
| 3.4. Riesgo 4 — Degradación de rendimiento.....                                                                  | 8  |
| 3.5. Riesgo 5 — Corrupción de datos .....                                                                        | 8  |
| 3.6. Riesgo 6 — Error humano en operaciones administrativas.....                                                 | 9  |
| 4. SECCIÓN III — RESOLUCIÓN DE 10 PROBLEMAS REALES.....                                                          | 9  |
| Problema 1 — "La base de datos está lenta de repente; consultas que antes volaban ahora tardan"<br>.....         | 9  |
| Problema 2 — "No se pueden insertar datos: el tablespace/disco está lleno" .....                                 | 10 |
| Problema 3 — "Las transacciones se quedan colgadas; hay bloqueos y deadlocks" .....                              | 10 |
| Problema 4 — "Necesitamos restaurar y el backup no sirve" .....                                                  | 10 |
| Problema 5 — "La aplicación reporta 'too many connections' / no hay conexiones disponibles"<br>.....             | 10 |
| Problema 6 — "La réplica está muy atrasada (replication lag) o se desincronizó" .....                            | 11 |
| Problema 7 — "El registro de transacciones (WAL / transaction log) creció sin control y llenó el<br>disco" ..... | 11 |
| Problema 8 — "Los índices están fragmentados y los planes de ejecución empeoraron".....                          | 12 |
| Problema 9 — "Sospechamos acceso no autorizado / hay una cuenta con privilegios excesivos"<br>.....              | 12 |
| Problema 10 — "Aparecen errores de bloque/página corrupta al leer una tabla" .....                               | 12 |
| 5. CONCLUSIONES .....                                                                                            | 13 |
| 6. REFERENCIAS.....                                                                                              | 14 |

## 1. INTRODUCCIÓN

Para casi cualquier organización, la base de datos es hoy uno de sus activos más críticos: en ella viven las transacciones, los clientes y la información que sostiene la operación diaria. De ahí la importancia del Administrador de Bases de Datos (DBA), encargado de que ese activo se mantenga disponible, íntegro, seguro y con buen rendimiento. En sistemas críticos como los de banca, salud o telecomunicaciones, un minuto de indisponibilidad puede significar pérdidas económicas, incumplimiento regulatorio y daño a la reputación; por eso el trabajo del DBA va mucho más allá de instalar un motor: abarca la continuidad del negocio, los respaldos, la seguridad, el rendimiento y la respuesta ante incidentes. Como señala Mullins (2012), es una disciplina técnica independiente del motor y orientada sobre todo a la prevención.

Para aterrizar el análisis incluyo ejemplos del sector bancario, inspirados en el tipo de entorno donde me desempeño (mainframe IBM Z con Db2 for z/OS, transacciones en línea por CICS y una ventana batch nocturna para el cierre de día).

## 2. SECCIÓN I — TAREAS CRÍTICAS QUE UN DBA DEBE EJECUTAR REGULARMENTE

### 2.1. Tarea 1 — Respaldo y verificación de copias de seguridad (Backup & Recovery)

**Propósito.** Garantizar que, ante cualquier evento de pérdida (fallo de hardware, error humano, ataque de ransomware, corrupción o desastre), la organización pueda restaurar la base de datos a un punto consistente, con la mínima pérdida de datos (RPO) y en el menor tiempo posible (RTO).

Conviene recordar que un respaldo solo cuenta como tal cuando se ha probado que se puede restaurar.

**Frecuencia típica.** Se combinan respaldos completos (diarios o semanales), incrementales o diferenciales (diarios o cada pocas horas) y respaldo continuo de los registros de transacciones (cada pocos minutos, para habilitar el *point-in-time recovery*). La verificación o restauración de prueba se hace de forma periódica, normalmente semanal o mensual.

**Consecuencias de no realizarla correctamente.** Pérdida total o parcial de datos sin posibilidad de recuperarlos, incumplimiento de obligaciones regulatorias de retención, paralización prolongada del negocio y, en el peor de los casos, el cierre de la operación. Es muy común que un backup que nunca se probó termine fallando precisamente el día que se necesita.

**Herramientas / comandos típicos.**

- **PostgreSQL:** `pg_dump` / `pg_dumpall` (lógico), `pg_basebackup` (físico), archivado de WAL para PITR, herramientas como pgBackRest o Barman.
- **Oracle:** RMAN (`BACKUP DATABASE PLUS ARCHIVELOG;`, `RESTORE/RECOVER`), Data Pump (`expdp/impdp`).
- **SQL Server:** `BACKUP DATABASE ... TO DISK = '...' WITH CHECKSUM;`, `BACKUP LOG, RESTORE VERIFYONLY`.

- **MySQL:** `mysqldump`, Percona XtraBackup, binary logs para PITR.
- **Db2 for z/OS:** utilidades `BACKUP SYSTEM / COPY` y `RECOVER`, con *image copies* y logs de Db2 para recuperación a un punto en el tiempo.

**Ejemplo en banca.** En una entidad financiera se programan *image copies* y respaldo continuo de los logs de transacciones, de modo que si un proceso del cierre de día deja un saldo en mal estado, se pueda recuperar la tabla afectada al instante previo al error (un RPO de pocos minutos). Cada cierto tiempo se hace además una restauración de prueba en un ambiente aislado, para confirmar que esas copias de verdad sirven y no descubrirlo recién durante una contingencia.

## 2.2. Tarea 2 — Monitoreo de rendimiento y optimización de consultas

**Propósito.** Asegurar que la base de datos responda dentro de los SLA, identificando y eliminando cuellos de botella (consultas lentas, contención de bloqueos, esperas de E/S, presión de memoria) antes de que el usuario los perciba. Incluye la observación de métricas clave: latencia, throughput, uso de CPU/memoria, *cache hit ratio*, consultas más costosas y eventos de espera (*wait events*).

**Frecuencia típica.** Continua o diaria mediante dashboards y alertas automáticas, y bajo demanda cuando se investiga un incidente o se despliega un cambio.

**Consecuencias de no realizarla correctamente.** Degradación progresiva del servicio, *timeouts* en las aplicaciones, experiencia de usuario deficiente, incumplimiento de SLA y, eventualmente, caídas por agotamiento de recursos. Los problemas de rendimiento no detectados a tiempo escalan a incidentes mayores.

### Herramientas / comandos típicos.

- **PostgreSQL:** extensión `pg_stat_statements`, vistas `pg_stat_activity`, `pg_stat_user_tables`, `EXPLAIN (ANALYZE, BUFFERS)`, herramientas como `pganalyze` o `PgHero`.
- **Oracle:** AWR (Automatic Workload Repository), ADDM, ASH, vistas `V$SESSION`, `V$SQL`, `EXPLAIN PLAN`.
- **SQL Server:** Query Store, Dynamic Management Views (`sys.dm_exec_query_stats`, `sys.dm_os_wait_stats`), Extended Events, plan de ejecución estimado/real.
- **MySQL:** `Performance Schema`, `sys` schema, `slow_query_log`, `EXPLAIN`.
- **Db2 for z/OS:** `EXPLAIN` con tablas de plan, *Accounting/Statistics traces*, monitores de rendimiento.

**Ejemplo en banca.** Durante el horario hábil, miles de cajeros, agencias y canales digitales usan al mismo tiempo las transacciones en línea a través de CICS. El DBA vigila la latencia de las operaciones más frecuentes (consulta de saldo, retiro, transferencia) y los tiempos de espera; si la consulta de un reporte nuevo empieza a afectar el tiempo de respuesta de los cajeros, lo ideal es detectarla y optimizarla antes de que el cliente lo note en ventanilla.

## 2.3. Tarea 3 — Mantenimiento de índices y estadísticas

**Propósito.** Conservar la salud física y lógica de los objetos: reorganizar/reconstruir índices fragmentados, recuperar espacio de filas borradas o actualizadas y, sobre todo, mantener actualizadas las estadísticas que el optimizador usa para elegir buenos planes de ejecución. En PostgreSQL esto incluye el **VACUUM/ANALYZE**, esencial para recuperar espacio y prevenir el *transaction ID wraparound* (PostgreSQL Global Development Group, 2024).

**Frecuencia típica.** En gran medida automatizada y continua (por ejemplo, el *autovacuum* de PostgreSQL o la actualización automática de estadísticas en SQL Server), complementada con ventanas de mantenimiento semanales o bajo demanda para reconstruir índices en tablas muy volátiles.

**Consecuencias de no realizarla correctamente.** Estadísticas obsoletas que llevan al optimizador a elegir planes ineficientes (escaneos completos en lugar de búsquedas por índice), *bloat* de tablas e índices que desperdicia disco y memoria, degradación creciente del rendimiento y, en PostgreSQL, riesgo de detención del motor por *wraparound* si nunca se ejecuta **VACUUM**.

**Herramientas / comandos típicos.**

- **PostgreSQL:** **VACUUM (ANALYZE) tabla;** **REINDEX**, configuración de *autovacuum*, vista **pg\_stat\_user\_tables** (columnas **n\_dead\_tup**, **last\_autovacuum**).
- **Oracle:** **DBMS\_STATS.GATHER\_TABLE\_STATS**, **ALTER INDEX ... REBUILD**.
- **SQL Server:** **UPDATE STATISTICS**, **ALTER INDEX ... REORGANIZE / REBUILD**, *maintenance plans*.
- **MySQL:** **ANALYZE TABLE**, **OPTIMIZE TABLE**.
- **Db2 for z/OS:** utilidad **RUNSTATS** (estadísticas para el optimizador) y **REORG** (reorganización de tablespaces e índices).

**Ejemplo en banca.** Las tablas con altísima volatilidad, como la de movimientos del día, se fragmentan rápido. El DBA programa **RUNSTATS/REORG** en la ventana batch nocturna para que, al abrir operaciones al día siguiente, el optimizador siga eligiendo accesos por índice y las transacciones de los cajeros no se degraden por estadísticas desactualizadas.

## 2.4. Tarea 4 — Gestión de seguridad, control de accesos y auditoría

**Propósito.** Proteger la confidencialidad e integridad de los datos aplicando el principio de mínimo privilegio: gestionar usuarios, roles y permisos, mantener el motor parchado, cifrar los datos en reposo y en tránsito, y registrar quién accede a qué (auditoría) para cumplir normativas como PCI-DSS, GDPR o leyes locales de protección de datos.

**Frecuencia típica.** La administración de cuentas y permisos se hace bajo demanda (altas, bajas y cambios); la revisión de privilegios y de los logs de auditoría es periódica (mensual o trimestral); y los parches de seguridad se aplican según el ciclo del proveedor y la criticidad de la vulnerabilidad.

**Consecuencias de no realizarla correctamente.** Brechas de seguridad con exfiltración de datos, fraude interno por privilegios excesivos, sanciones regulatorias, multas, demandas y pérdida de

confianza de los clientes. Una sola cuenta sobreprivilegiada o un motor sin parchar puede comprometer toda la organización.

#### Herramientas / comandos típicos.

- **SQL estándar:** `CREATE ROLE`, `GRANT/REVOKE`, `CREATE USER ... WITH PASSWORD ....`
- **PostgreSQL:** roles y `pg_hba.conf`, extensión `pgaudit`, cifrado TLS.
- **Oracle:** *Database Vault*, *Audit Vault*, *Transparent Data Encryption (TDE)*, perfiles de contraseña.
- **SQL Server:** *roles*, *SQL Server Audit*, *Transparent Data Encryption*, *Always Encrypted*.
- **MySQL:** `mysql_secure_installation`, plugin de auditoría, SSL/TLS.
- **Db2 for z/OS:** integración con RACF para autenticación/autorización, cifrado de datos y *audit policies*.

**Ejemplo en banca.** El acceso a los datos de clientes está regido por el principio de mínimo privilegio y normativas como PCI-DSS: la cuenta de una aplicación de consulta no debería poder modificar saldos, y toda lectura sensible queda registrada en auditoría. Periódicamente se revisan privilegios y bitácoras para detectar accesos anómalos, y los accesos administrativos exigen segundo factor.

## 2.5. Tarea 5 — Gestión de capacidad y crecimiento del almacenamiento

**Propósito.** Anticipar y administrar el crecimiento de los datos, los archivos de datos, los *tablespaces*, los registros de transacciones (WAL/redo/transaction log) y el espacio temporal, de modo que nunca falte disco ni se degrade el rendimiento por falta de recursos. Incluye la planificación de capacidad a futuro con base en tendencias históricas.

**Frecuencia típica.** Monitoreo diario o continuo del espacio libre con alertas por umbral; análisis de tendencias y planificación de capacidad cada mes o trimestre; y archivado o purga de datos históricos según la política de retención.

**Consecuencias de no realizarla correctamente.** Si un *tablespace*, el disco o el registro de transacciones se llena, el motor deja de aceptar escrituras o se detiene por completo, lo que provoca una caída total del servicio. También aparece degradación por contención de E/S y costos innecesarios cuando se sobreaprovisiona sin un buen análisis.

#### Herramientas / comandos típicos.

- **PostgreSQL:** `pg_database_size()`, `pg_relation_size()`, monitoreo del directorio `pg_wal`, *tablespace*.
- **Oracle:** vistas `DBA_DATA_FILES`, `DBA_FREE_SPACE`, `DBA_TABLESPACE_USAGE_METRICS`, *datafiles* con `AUTOEXTEND`.
- **SQL Server:** `sp_spaceused`, `sys.dm_db_file_space_usage`, *autogrowth* configurado con incrementos fijos.
- **General:** integración con herramientas de monitoreo de infraestructura (Prometheus/Grafana, Zabbix, Nagios, Datadog) para alertas de disco.

- **Db2 for z/OS:** monitoreo de tablespaces y de la utilización de los *active/archive logs*.

**Ejemplo en banca.** El cierre de fin de mes genera un volumen de movimientos varias veces mayor al de un día normal. El DBA debe anticipar ese pico para que ni el almacenamiento ni el log de transacciones se llenen durante la ventana batch: si el log se llena a mitad del cierre, el proceso se detiene y la apertura de operaciones del día siguiente se retrasa. Por eso se planifica capacidad con base en las tendencias históricas y se archivan/purgan datos según la política de retención.

## 3. SECCIÓN II — RIESGOS SIGNIFICATIVOS EN LA ADMINISTRACIÓN DE UN DBMS

### 3.1. Riesgo 1 — Pérdida de datos por fallo o ausencia de respaldo válido

- **Descripción técnica.** Imposibilidad de recuperar datos por inexistencia de respaldos, respaldos corruptos, incompletos o nunca probados, o una cobertura de respaldo que no cumple el RPO definido.
- **Escenario concreto.** Un disco del servidor de producción falla físicamente; al intentar restaurar, se descubre que el job de backup llevaba tres semanas fallando en silencio y el último respaldo válido está corrupto.
- **Impacto.** Pérdida de datos permanente, paralización del negocio, incumplimiento regulatorio de retención, posibles demandas.
- **Control / práctica de mitigación.** Estrategia 3-2-1 (tres copias, en dos medios distintos, una fuera de sitio), pruebas de restauración periódicas, monitoreo y alertas sobre el éxito/fallo de cada job de backup, y verificación de integridad (**CHECKSUM, RESTORE VERIFYONLY**).

### 3.2. Riesgo 2 — Tiempo de inactividad (downtime) no planificado

- **Descripción técnica.** Indisponibilidad del servicio de base de datos por fallo de hardware, caída del nodo, agotamiento de recursos, error de configuración o mantenimiento mal ejecutado.
- **Escenario concreto.** El único servidor de base de datos (sin réplica) sufre una falla de memoria a las 10:00 a.m. de un día hábil; la aplicación de ventas queda fuera de línea durante horas mientras se consigue y aprovisiona hardware.
- **Impacto.** Pérdida de ingresos por minuto caído, incumplimiento de SLA, daño reputacional, sobrecarga al restablecer el servicio.
- **Control / práctica de mitigación.** Arquitectura de alta disponibilidad (replicación, *clustering* y *failover* automático), redundancia de hardware, un plan de recuperación ante desastres documentado y probado, y la eliminación de puntos únicos de falla.

**Ejemplo en banca.** Para un banco, la indisponibilidad del core en horario hábil significa cajeros, agencias y banca en línea fuera de servicio simultáneamente. Por eso estos entornos operan con alta disponibilidad y un sitio alterno de contingencia, de modo que ante la caída de un componente el servicio conmute con interrupción mínima.

### 3.3. Riesgo 3 — Brecha de seguridad y acceso no autorizado

- **Descripción técnica.** Acceso indebido a los datos por credenciales débiles o filtradas, privilegios excesivos, ausencia de cifrado, inyección SQL en la capa de aplicación o motores sin parchar.
- **Escenario concreto.** Una cuenta de aplicación con permisos de administrador es comprometida mediante una inyección SQL; el atacante extrae la tabla completa de clientes con datos personales y tarjetas.
- **Impacto.** Exfiltración de datos sensibles, fraude, multas regulatorias (GDPR/PCI-DSS), litigios y pérdida de confianza.
- **Control / práctica de mitigación.** Aplicar mínimo privilegio, usar consultas parametrizadas, cifrar en reposo y en tránsito (TDE/TLS), exigir MFA para accesos administrativos, parchar a tiempo, mantener la auditoría activa y segmentar la red.

### 3.4. Riesgo 4 — Degradación de rendimiento

- **Descripción técnica.** Deterioro gradual o súbito del tiempo de respuesta por consultas no optimizadas, estadísticas obsoletas, falta de índices (o exceso de ellos), *bloat*, contención de bloqueos o saturación de recursos.
- **Escenario concreto.** Tras un crecimiento de 5x en el volumen de datos, una consulta de reporte que tomaba 2 segundos ahora tarda 4 minutos porque el optimizador dejó de usar el índice por estadísticas desactualizadas.
- **Impacto.** *Timeouts* de aplicación, mala experiencia de usuario, incumplimiento de SLA, efecto cascada que puede tumbar el servicio.
- **Control / práctica de mitigación.** Monitoreo continuo y *baselines* de rendimiento, mantenimiento de estadísticas e índices, *tuning* de consultas con **EXPLAIN**, pruebas de carga antes de producción y planificación de capacidad.

### 3.5. Riesgo 5 — Corrupción de datos

- **Descripción técnica.** Daño físico o lógico de bloques de datos por fallos de hardware (disco/memoria/controladora), bugs del motor, cortes de energía sin protección o errores del sistema de archivos.
- **Escenario concreto.** Un corte eléctrico abrupto sin UPS deja varios bloques de datos en estado inconsistente; al leer una tabla aparece el error de "bloque corrupto" y parte de la información es ilegible.
- **Impacto.** Pérdida de integridad, resultados erróneos en operaciones críticas, posible pérdida de datos y downtime durante la reparación.
- **Control / práctica de mitigación.** Hardware confiable con ECC y UPS, suma de verificación de páginas (**data\_checksums** / **PAGE\_VERIFY CHECKSUM**), verificaciones periódicas de integridad (**DBCC CHECKDB**, **pg\_amcheck**), redundancia de almacenamiento (RAID) y respaldos que permitan PITR.

### 3.6. Riesgo 6 — Error humano en operaciones administrativas

- **Descripción técnica.** Acciones destructivas o erróneas ejecutadas por personas: un **DELETE/UPDATE** sin **WHERE**, un **DROP TABLE** en el ambiente equivocado, un cambio de configuración mal aplicado o un despliegue sin revisión.
- **Escenario concreto.** Un DBA cree estar conectado al ambiente de pruebas y ejecuta **UPDATE cuentas SET saldo = 0;** contra producción, afectando a todos los clientes.
- **Impacto.** Pérdida o alteración masiva de datos, downtime, retrabajo y necesidad de restauración desde backup.
- **Control / práctica de mitigación.** Separación estricta de ambientes con indicadores visuales, control de cambios y revisión por pares, ejecución dentro de transacciones (**BEGIN ... ROLLBACK/COMMIT**), uso de cuentas de menor privilegio para tareas rutinarias, automatización de despliegues y respaldos previos a cambios mayores.

**Ejemplo en banca.** En una entidad financiera, un cambio en producción casi nunca lo aplica una sola persona a su criterio: pasa por una gestión de cambios y una segunda revisión, justamente para evitar que un comando ejecutado en el ambiente equivocado afecte saldos reales de clientes.

## 4. SECCIÓN III — RESOLUCIÓN DE 10 PROBLEMAS REALES

En esta sección se analizan diez problemas reales y de alta frecuencia en producción.

### Problema 1 — "La base de datos está lenta de repente; consultas que antes volaban ahora tardan"

- **Causa más probable.** Estadísticas desactualizadas tras una carga masiva de datos, lo que provoca que el optimizador elija planes deficientes (p. ej. *full scan* en lugar de índice). El cambio de volumen invalidó el plan que antes era óptimo.
- **Solución concreta.** Identificar la consulta con **pg\_stat\_statements**/Query Store, analizar su plan con **EXPLAIN (ANALYZE, BUFFERS)** y actualizar estadísticas:

```
-- PostgreSQL
ANALYZE VERBOSE ventas;
-- Oracle
EXEC DBMS_STATS.GATHER_TABLE_STATS('APP', 'VENTAS');
-- SQL Server
UPDATE STATISTICS dbo.Ventas WITH FULLSCAN;
```

Si falta un índice de apoyo, crearlo: **CREATE INDEX idx\_ventas\_fecha ON ventas(fecha);**

- **Acción preventiva.** Garantizar que el *autovacuum*/*auto-update statistics* esté habilitado y bien configurado, y ejecutar **ANALYZE** automáticamente tras cargas masivas (ETL).

## Problema 2 — "No se pueden insertar datos: el tablespace/disco está lleno"

- **Causa más probable.** Crecimiento no monitoreado de datos o de objetos temporales, o un *datafile* sin **AUTOEXTEND**, que agotó el espacio disponible.
- **Solución concreta.** Liberar o ampliar espacio de inmediato:

```
-- Oracle: agregar datafile o habilitar autoextend
ALTER TABLESPACE datos ADD DATAFILE '/u02/datos02.dbf' SIZE 5G AUTOEXTEND ON;
-- PostgreSQL: recuperar espacio de bloat y/o mover a otro tablespace
VACUUM (FULL) tabla_inflada;
```

En el sistema operativo, purgar logs/archivos temporales y, si aplica, ampliar el volumen.

- **Acción preventiva.** Alertas por umbral de espacio (p. ej. al 80 %), política de archivado/purga de datos históricos y planificación de capacidad basada en tendencias.

## Problema 3 — "Las transacciones se quedan colgadas; hay bloqueos y deadlocks"

- **Causa más probable.** Transacciones largas que retienen bloqueos, orden inconsistente de acceso a recursos entre sesiones o ausencia de índices que obliga a bloquear más filas de las necesarias.
- **Solución concreta.** Detectar y resolver el bloqueo:

```
-- PostgreSQL: ver bloqueos y sesión bloqueadora
SELECT pid, state, wait_event, query FROM pg_stat_activity WHERE wait_event_type =
'Lock';
SELECT pg_terminate_backend(<pid_bloqueador>); -- como último recurso
-- SQL Server
SELECT * FROM sys.dm_tran_locks; -- y revisar el deadlock graph en Extended Events
```

- **Acción preventiva.** Mantener transacciones cortas, acceder a los objetos siempre en el mismo orden, crear los índices adecuados, definir un **lock\_timeout** razonable y reintentar en la aplicación ante deadlocks.

## Problema 4 — "Necesitamos restaurar y el backup no sirve"

- **Causa más probable.** Jobs de respaldo que fallaban sin ser notificados o respaldos nunca verificados con una prueba de restauración real.
- **Solución concreta.** Recuperar desde el último respaldo válido y, si existe archivado de logs, aplicar PITR hasta el último punto consistente:

```
-- SQL Server: validar antes de confiar
RESTORE VERIFYONLY FROM DISK = 'D:\bk\app_full.bak';
RESTORE DATABASE App FROM DISK='...full.bak' WITH NORECOVERY;
RESTORE LOG App FROM DISK='...log.trn' WITH RECOVERY, STOPAT='2026-06-06 09:59:00';
```

- **Acción preventiva.** Probar restauraciones periódicamente en un ambiente aislado, monitorear y alertar el resultado de cada backup, usar **WITH CHECKSUM** y aplicar la estrategia 3-2-1.

## Problema 5 — "La aplicación reporta 'too many connections' / no hay conexiones disponibles"

- **Causa más probable.** Ausencia de un *connection pool* (o mal dimensionado), conexiones que no se cierran (*leaks*) o **max\_connections** configurado por debajo de la demanda real.
- **Solución concreta.** Liberar conexiones inactivas y/o subir el límite temporalmente:

```
-- PostgreSQL: terminar sesiones idle prolongadas
SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE state='idle' AND now()-state_change > interval '30 min';
SHOW max_connections;
```

Implementar un pooler como PgBouncer delante del motor.

- **Acción preventiva.** Usar *connection pooling* en la aplicación, configurar **idle\_in\_transaction\_session\_timeout**, dimensionar correctamente el pool y monitorear el número de conexiones activas.

## Problema 6 — "La réplica está muy atrasada (replication lag) o se desincronizó"

- **Causa más probable.** Saturación de E/S o red en el primario/réplica, una transacción muy pesada o un conflicto/consulta larga en la réplica que bloquea la aplicación de cambios.
- **Solución concreta.** Medir el retraso y atacar la causa:

```
-- PostgreSQL en la réplica
SELECT now() - pg_last_xact_replay_timestamp() AS lag;
-- MySQL
SHOW REPLICA STATUS\G -- revisar Seconds_Behind_Source
```

Aliviar la carga del primario, optimizar la transacción pesada y, si la réplica quedó irrecuperable, reconstruirla desde un *base backup*.

- **Acción preventiva.** Monitorear el lag con alertas, dimensionar adecuadamente la red y el almacenamiento, y evitar consultas analíticas pesadas sobre réplicas que deben mantenerse al día.

## Problema 7 — "El registro de transacciones (WAL / transaction log) creció sin control y llenó el disco"

- **Causa más probable.** Un *replication slot* inactivo o un proceso de archivado detenido que impide reciclar el WAL, una transacción abierta de muy larga duración, o el modo de recuperación *FULL* sin respaldos de log frecuentes (en SQL Server).
- **Solución concreta.**

```
-- SQL Server: respaldar el log para truncarlo (no usar SHRINK como hábito)
BACKUP LOG App TO DISK='D:\bk\app_log.trn';
-- PostgreSQL: detectar slots inactivos que retienen WAL
SELECT slot_name, active, restart_lsn FROM pg_replication_slots;
SELECT pg_drop_replication_slot('slot_huerfano');
```

Cerrar transacciones colgadas y restablecer el proceso de archivado.

- **Acción preventiva.** Respaldos de log frecuentes en modo *FULL*, monitoreo del crecimiento del WAL/log, control de transacciones de larga duración y vigilancia de *replication slots* huérfanos.

## Problema 8 — "Los índices están fragmentados y los planes de ejecución empeoraron"

- **Causa más probable.** Alta volatilidad de datos (muchos **INSERT/UPDATE/DELETE**) que fragmenta los índices y genera *bloat*, reduciendo la eficiencia del acceso.
- **Solución concreta.**

```
-- SQL Server: reorganizar o reconstruir según el nivel de fragmentación
ALTER INDEX ALL ON dbo.Pedidos REBUILD;
-- PostgreSQL
REINDEX TABLE CONCURRENTLY pedidos;
```

- **Acción preventiva.** Plan de mantenimiento que reorganice (10–30 % de fragmentación) o reconstruya (más del 30 %) los índices según corresponda, *fill factor* adecuado en tablas muy volátiles y *autovacuum* bien afinado.

## Problema 9 — "Sospechamos acceso no autorizado / hay una cuenta con privilegios excesivos"

- **Causa más probable.** Violación del principio de mínimo privilegio: cuentas de aplicación o de usuario con permisos administrativos que no necesitan, sumado a la falta de auditoría.
- **Solución concreta.** Revisar y revocar privilegios, rotar credenciales y activar auditoría:

```
-- Revisar y ajustar privilegios
REVOKE ALL PRIVILEGES ON DATABASE app FROM usuario_app;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO usuario_app;
ALTER ROLE usuario_app WITH PASSWORD 'nueva_clave_fuerte';
```

Habilitar **pgaudit**/SQL Server Audit y revisar logs en busca de accesos anómalos.

- **Acción preventiva.** Aplicar mínimo privilegio por diseño, usar roles, rotación periódica de credenciales, MFA para accesos administrativos, parametrizar consultas (anti-inyección) y auditar de forma continua.

## Problema 10 — "Aparecen errores de bloque/página corrupta al leer una tabla"

- **Causa más probable.** Corrupción física por fallo de disco/memoria o un corte de energía abrupto que dejó páginas inconsistentes.
- **Solución concreta.** Diagnosticar el alcance y restaurar la parte afectada:

```
-- SQL Server: verificar integridad
DBCC CHECKDB ('App') WITH NO_INFOMSGS, ALL_ERRORMSG;
-- restaurar una página específica desde backup si es viable
RESTORE DATABASE App PAGE='1:57837' FROM DISK='...full.bak' WITH NORECOVERY;
```

En PostgreSQL, verificar con **pg\_amcheck** y restaurar/PITR desde un respaldo sano; reemplazar el hardware defectuoso.

- **Acción preventiva.** Activar sumas de verificación de páginas (**data\_checksums/PAGE\_VERIFY CHECKSUM**), usar hardware con ECC y UPS, RAID redundante, y ejecutar verificaciones de integridad periódicas junto con respaldos que permitan PITR.

## 5. CONCLUSIONES

Tras analizar las tareas, los riesgos y los problemas anteriores, los aprendizajes que rescato son:

1. Prevenir sale más barato que recuperar: casi todos los incidentes graves se evitan antes de la crisis, con respaldos probados, monitoreo y mantenimiento constante.
2. Un respaldo solo vale si se ha probado restaurarlo; por eso la estrategia de backup y el cumplimiento del RPO/RTO son la base de la continuidad del negocio.
3. Las tareas críticas están conectadas: el rendimiento depende del mantenimiento, la disponibilidad de la capacidad y la alta disponibilidad, y la seguridad atraviesa a todas; descuidar una aparece como problema en otra.
4. Saber el comando no basta: el buen DBA sabe sobre todo cuándo usarlo, mide el impacto y reduce el error humano con buenas prácticas y control de cambios.
5. Administrar bases de datos es, en el fondo, gestionar riesgos: cada decisión es una valoración del riesgo aceptable frente a lo crítico que sea el sistema.

## 6. REFERENCIAS

(Formato APA, 7.<sup>a</sup> edición.)

- Connolly, T., & Begg, C. (2015). *Database systems: A practical approach to design, implementation, and management* (6th ed.). Pearson.
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.
- EnterpriseDB. (2020). *PostgreSQL 13: Don't let replication slots kill your primary*. EDB Blog. <https://www.enterprisedb.com/blog/postgresql-13-dont-let-slots-kill-your-primary>
- Microsoft. (2024). *Back up and restore of SQL Server databases*. Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/backup-restore/>
- Microsoft. (2024). *Reorganize and rebuild indexes — SQL Server*. Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/indexes/reorganize-and-rebuild-indexes>
- Microsoft. (2024). *SQL Server deadlocks guide*. Microsoft Learn. <https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-deadlocks-guide>
- Mullins, C. S. (2012). *Database administration: The complete guide to DBA practices and procedures* (2nd ed.). Addison-Wesley Professional.
- Oracle. (2024). *Oracle Database administrator's guide, 19c*. Oracle Corporation. <https://docs.oracle.com/en/database/oracle/oracle-database/19/admin/>
- Percona. (2021). *Overcoming VACUUM wraparound*. Percona Database Performance Blog. <https://www.percona.com/blog/overcoming-vacuum-wraparound/>
- PgBouncer Project. (2024). *PgBouncer — Lightweight connection pooler for PostgreSQL: Configuration*. <https://www.pgbouncer.org/config.html>
- PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation — Chapter 25: Routine database maintenance tasks*. <https://www.postgresql.org/docs/16/routine-vacuuming.html>
- Veeam. (2023). *The 3-2-1 backup rule explained*. Veeam Blog. <https://www.veeam.com/blog/321-backup-rule.html>